COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	10/16/2026	**DATE SUBMITTED:**	10/16/2026
NAME:	Barizo, Julian Benedict D.R. Macatangay Justin Jake B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 13.1 **[Parallel Algorithms and Multithreading]**

1. PROCEDURE OUTPUT

ILO A

Single Thread

Code

```cpp
#include <iostream>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();

return 0;
}
```

Output

```
Printing integer: 10
Printing string: T.I.P.
```

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis

For this code, it utilizes a single thread with the name **t1** where it runs a print function along the given parameters 10 and “T.I.P”. Then, the main thread calls t1.join() for the thread to wait, pausing the execution until the worker thread completes the task and prints the values.

T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

ILO B

Multithread

Code

HOA13.1 > ILO_B.cpp > main()

```cpp
#include <iostream>
#include <thread>
#include <string>
#include <vector>


void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}




int main() {
    std::vector<std::string> s = {
    "T.I.P.",
    "Competent",
    "Computer",
    "Engineers"
};


std::vector<std::thread> threads;
    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
}
    for (auto &th : threads) {
        th.join();
}
return 0;
}
```

Output

```
0 : T.I.P.3 : Engineers
1 : Competent
2 : Computer
```

```
0 : T.I.P.3 : Engineers1 : Competent


2 : Computer
```

```
0 : T.I.P.
2 : Computer
1 : Competent3 : Engineers
```

Analysis

The multithreaded program iterates through a vector of strings to spawn multiple concurrent std::thread objects into a vector container where each thread formats its message before writing to std::cout and subsequently uses a second loop to join() all threads, demonstrating parallel execution across CPU cores where output order may vary based on thread scheduling.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A

1.) Write a definition of multithreading and its advantages/disadvantages.

Barizo	Multithreading is a CPU execution technique that allows a single process to spawn multiple concurrent threads of execution that share memory and resources. Its primary advantages are enhanced application responsiveness and efficient resource utilization across multi-core processors. However, its disadvantages include increased system complexity and potential software bugs, such as race conditions, deadlocks, and synchronization overhead.
Macatangay	Multithreading executes multiple thread paths concurrently within a shared memory process to maximize multi-core CPU usage and keep application user interfaces responsive, though it introduces risks of data races, complex debugging, and context-switching overhead.

2.) Rationalize the use of multithreading by providing at least 3 use-cases

Barizo	Multithreading is essential for optimizing system efficiency by executing independent tasks concurrently rather than sequentially. Common use-cases include web servers handling thousands of simultaneous user requests, video games rendering graphics on one thread while computing physics on another, and web browsers loading distinct UI components, scripts, and media files simultaneously. By distributing workload across multiple threads, applications prevent UI freezing and maximize processor throughput.
Macatangay	Multithreading keeps graphical user interfaces responsive during heavy computations, allows web servers to process thousands of requests concurrently, and enables video game engines to run physics, rendering, and sound systems simultaneously.
3.) Differentiate between parallelism and concurrency.	
Barizo	Concurrency is about dealing with multiple tasks at once by interleaving their execution on a system, whereas parallelism is about doing multiple tasks at the exact same physical instant using multiple CPU cores.
Macatangay	Concurrency manages multiple overlapping tasks on a shared process through task switching, whereas parallelism physically executes independent tasks at the exact same instant across separate hardware CPU cores.

Part B

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

- Create a global variable of type integer.
- Create an add function that will take an integer parameter and add that value to the global variable.
- Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
- Display the value of global variable at different combinations of using the join() per thread. Such that:
  - Display
  - T1.join()
  - Display
  - T2.join()
  - Display
  - T3.join()
- Provide an analysis based on the outputs of the multi-threading exercise.

Code

HOA12.1 > PartB.cpp > ...

```cpp
#include <iostream>
#include <thread>
#include <chrono>

int globalValue = 0;

void add(int value) {
    std::this_thread::sleep_for(std::chrono::milliseconds(value * 20)); // small delay so the joins are visible
    globalValue += value;
}

int main() {
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    std::cout << "Display: " << globalValue << std::endl;
    T1.join();
    std::cout << "Display: " << globalValue << std::endl;
    T2.join();
    std::cout << "Display: " << globalValue << std::endl;
    T3.join();
    std::cout << "Display: " << globalValue << std::endl;

    return 0;
}

```

Output

```
artB } ; if ($?) { .\PartB }
Display: 0
Display: 10
Display: 30
Display: 60
```

```
PS C:\Users\Justin Macatangay\Documents\Macat
cpp -o PartB } ; if ($?) { .\PartB }
Display: 0
Display: 10
Display: 30
Display: 60
PS C:\Users\Justin Macatangay\Documents\Macat
```

Analysis

The initial value displays as 0 because main prints before any thread finishes updating the global variable. Calling T1.join() forces the program to wait until the first thread completes, updating the total to 10. Subsequent join() calls sequentially synchronize each remaining thread, showing the cumulative updates up to the final total of 60. This demonstrates how joining threads ensures proper execution ordering and consistent output when modifying shared variables.

Part C

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

(include screenshots)

Code - Merge Sort

HOA12.1 > PartC.cpp > ...

```cpp
#include <iostream>
#include <thread>
#include "sortingAlgo2.h"

int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10, 55, 1, 67, 22, 90, 14, 5, 76, 31};
    int n = sizeof(arr) / sizeof(arr[0]);
    int mid = (n - 1) / 2;

    std::cout << "Before sorting:" << std::endl;
    printArray(arr, n);

    // one thread per half
    std::thread t1(mergeSort<int>, arr, 0, mid);
    std::thread t2(mergeSort<int>, arr, mid + 1, n - 1);
    t1.join();
    t2.join();

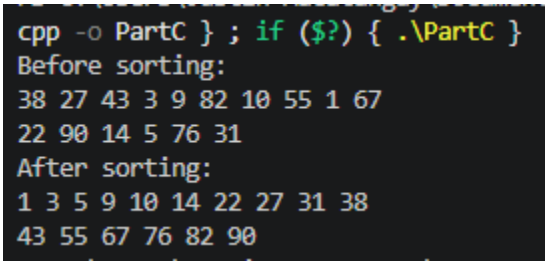
    // combine the two sorted halves
    merge(arr, 0, mid, n - 1);

    std::cout << "After sorting:" << std::endl;
    printArray(arr, n);

    return 0;
}

```

Output



Analysis

The code optimizes merge sort from our past HOA by using multi-threading to split the sorting workload across CPU cores. Thread t1 sorts the left half of the array while thread t2 simultaneously sorts the right half. Joining both threads ensures both halves are fully sorted before calling merge() to combine them into a single ordered sequence. This parallel approach improves execution efficiency for large datasets by utilizing hardware concurrency.

Conclusion

Barizo

I learnt about multithreading and how it allows multiple tasks to run concurrently to improve program efficiency and responsiveness. I also learned the difference between concurrency and parallelism, as well as how thread synchronization using join() ensures that threads finish their tasks before the program proceeds. Through this activity, I understood how multiple threads can work with shared variables and why proper synchronization is important for producing consistent results. Lastly, I learned how multithreading can optimize algorithms like merge sort by dividing the workload between threads, allowing different parts of the array to be sorted simultaneously.

Macatangay

This activity was very useful for understanding the difference and application of concurrency and parallelism with concurrent execution using multiple threads in C++. This approach can involve performing tasks in parallel, such as breaking up the work in the merge sort into multiple threads, to enhance the efficiency of computation and speed of execution. The step-by-step synchronization with thread joining showed the critical importance of controlling execution flow in order to have data consistency when operating on shared memory locations. Finally, using multi-threading techniques in secondhand to improve performance and resource allocation with modern multi-core computing systems.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Generation, Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.